📚 TEMA 03: Valores Aleatórios e Laços de Repetição

🎯 Habilidades Desenvolvidas

•Codificar programas utilizando técnica de programação estruturada
•Depurar e versionar programas, utilizando ambiente de desenvolvimento integrado

---

1. VALORES ALEATÓRIOS

Os valores aleatórios são muito utilizados por programadores em diversas situações. As linguagens de programação normalmente possuem funções para a criação de valores aleatórios e através delas podemos gerar valores para utilizarmos na criação de, por exemplo: jogos, senhas, captcha, algoritmos de criptografia, etc.

1.1 Biblioteca Util

No Portugol Webstudio, para gerar números aleatórios, precisamos importar a **Biblioteca Util**. Essa biblioteca contém funções especiais para trabalhar com números aleatórios, pausas no programa e medição de tempo.

Para importar uma biblioteca, usamos o comando:

```
inclua biblioteca Util
```

1.2 Números Aleatórios com Util.sorteia()

A função `Util.sorteia(menor, maior)` gera um número inteiro aleatório dentro do intervalo que você definir.

Sintaxe:

```
numero = Util.sorteia(1, 100)
```

Exemplo completo:

```
programa {
    inclua biblioteca Util

    funcao inicio() {
        inteiro numero_aleatorio

        numero_aleatorio = Util.sorteia(1, 100)

        escreva("Número aleatório gerado: ", numero_aleatorio)
    }
}
```

📌 **Explicação:**

•**Linha 2:** Importa a biblioteca Util
•**Linha 6:** Gera um número entre 1 e 100
•**Linha 8:** Exibe o número gerado

1.3 Gerando Números para Loteria

Para gerar números de 1 a 60 (como na Mega-Sena), basta ajustar os parâmetros:

```
programa {
    inclua biblioteca Util

    funcao inicio() {
        inteiro numero

        numero = Util.sorteia(1, 60)
        escreva("Número sorteado: ", numero)
    }
}
```

1.4 Pausando o Programa com Util.aguarde()

A função `Util.aguarde(milissegundos)` faz o programa esperar pelo tempo especificado. É muito útil para criar animações ou dar tempo para o usuário ler mensagens.

Sintaxe:

```
Util.aguarde(2000)  // Aguarda 2 segundos (2000 milissegundos)
```

Exemplo:

```
programa {
    inclua biblioteca Util

    funcao inicio() {
        inteiro inicio, fim

        inicio = Util.tempo_decorrido()
        escreva("Aguardando 2 segundos...\n")
        Util.aguarde(2000)
        fim = Util.tempo_decorrido()

        escreva("Tempo decorrido: ", fim - inicio, " ms")
    }
}
```

📌 **Explicação:**

•**Linha 7:** Marca o tempo inicial
•**Linha 9:** Pausa o programa por 2 segundos
•**Linha 10:** Marca o tempo final
•**Linha 12:** Calcula e exibe o tempo total

---

2. LAÇOS DE REPETIÇÃO

Os laços de repetição são muito utilizados nos sistemas para fazer algum tipo de consistência de dados do usuário. Ocorre que, por engano ou distração, o usuário pode entrar com informações inválidas que, se não forem consistidas, podem prejudicar a base de dados dos sistemas.

2.1 Laço ENQUANTO (Pré-Testado)

Neste tipo de laço, **enquanto** a condição estiver sendo atendida, o programa permanecerá em looping. O teste é feito **antes** de executar o bloco.

Sintaxe:

```
enquanto (condicao) {
    // Instruções a serem repetidas
}
```

Exemplo - Contagem de 1 a 5:

```
programa {
    funcao inicio() {
        inteiro contador = 1

        enquanto (contador <= 5) {
            escreva("Repetição número: ", contador, "\n")
            contador = contador + 1
        }

        escreva("Fim do laço!")
    }
}
```

📌 **Funcionamento:**

1.O programa verifica se `contador <= 5`
2.Se for verdadeiro, executa o bloco e incrementa o contador
3.Volta ao passo 1
4.Quando a condição for falsa, sai do laço

2.2 Laço FAÇA-ENQUANTO (Pós-Testado)

Neste tipo de laço, o algoritmo **primeiro faz** algo, **depois verifica** a condição. Isso garante que o bloco seja executado pelo menos uma vez.

Sintaxe:

```
faca {
    // Instruções a serem repetidas
} enquanto (condicao)
```

Exemplo - Validação de entrada:

```
programa {
    funcao inicio() {
        inteiro valor
```

```
        faca {
            escreva("Digite um número positivo: ")
            leia(valor)
            se (valor <= 0) {
                escreva("❌ Valor inválido! Tente novamente.\n")
            }
        } enquanto (valor <= 0)

        escreva("✅ Valor válido: ", valor)
    }
}
```

📌 **Funcionamento:**

1.Executa o bloco pelo menos uma vez
2.Depois verifica a condição
3.Se for verdadeira, repete o bloco
4.Se for falsa, sai do laço

2.3 Laço PARA (Com Variável de Controle)

Neste tipo de laço, há uma variável que funcionará como contador. Já na definição do laço, é especificado o número de repetições. A cada repetição, a variável contadora recebe automaticamente o acréscimo.

Sintaxe:

```
para (inteiro contador = 1; contador <= 10; contador++) {
    // Instruções a serem repetidas
}
```

Exemplo - Tabuada:

```
programa {
    funcao inicio() {
        inteiro numero = 3

        para (inteiro i = 1; i <= 10; i++) {
            escreva(numero, " x ", i, " = ", numero * i, "\n")
        }
    }
}
```

📌 **Explicação da estrutura:**

•`inteiro i = 1` → **Inicialização:** começa com 1
•`i <= 10` → **Condição:** repete enquanto for verdadeira
•`i++` → **Incremento:** aumenta 1 a cada repetição

---

3. EXERCÍCIOS RESOLVIDOS

🎮 Exercício 1: Desafios Aleatórios

Enunciado: Desenvolva um programa de desafios para o usuário. O desafio dependerá do valor aleatório gerado entre 0 e 9. Utilize a condicional "Escolha Caso" para definir os desafios.

Solução:

```
programa {
    inclua biblioteca Util

    funcao inicio() {
        inteiro numero

        numero = Util.sorteia(0, 9)

        escreva("Número sorteado: ", numero, "\n")
        escreva("=== SEU DESAFIO ===\n")

        escolha(numero) {
            caso 0:
                escreva("Faça 10 polichinelos!")
                pare
            caso 1:
                escreva("Cante uma música!")
                pare
            caso 2:
                escreva("Diga uma palavra em inglês!")
                pare
            caso 3:
                escreva("Faça uma careta engraçada!")
```

```
                pare
            caso 4:
                escreva("Conte uma piada!")
                pare
            caso 5:
                escreva("Imite um animal!")
                pare
            caso 6:
                escreva("Dê 5 pulos!")
                pare
            caso 7:
                escreva("Fale um trava-língua!")
                pare
            caso 8:
                escreva("Faça um cumprimento diferente!")
                pare
            caso 9:
                escreva("Deseje bom dia para alguém!")
                pare
            caso contrario:
                escreva("Desafio especial: Sorria!")
        }
    }
}
```

📌 **Explicação:**

•Gera um número aleatório entre 0 e 9
•Usa `escolha` com 10 casos diferentes
•Cada caso exibe um desafio diferente
•O `pare` evita que execute os casos seguintes

🎲 Exercício 2: Gerador de Loteria

Enunciado: Desenvolva um programa para gerar 6 números aleatórios para serem jogados na loteria. Os números devem estar no intervalo de 1 a 60, sem repetição.

Solução:

```
programa {
    inclua biblioteca Util
```

```
    funcao inicio() {
        inteiro numeros[6]
        inteiro i, j, temp
        logico repetido

        escreva("=== NÚMEROS DA LOTERIA ===\n")
        escreva("Gerando 6 números entre 1 e 60...\n\n")

        // Gera 6 números únicos
        para (i = 0; i < 6; i++) {
            faca {
                numeros[i] = Util.sorteia(1, 60)
                repetido = falso

                // Verifica se o número já foi sorteado
                para (j = 0; j < i; j++) {
                    se (numeros[i] == numeros[j]) {
                        repetido = verdadeiro
                    }
                }
            } enquanto (repetido == verdadeiro)
        }

        // Ordena os números (ordem crescente)
        para (i = 0; i < 5; i++) {
            para (j = i + 1; j < 6; j++) {
                se (numeros[i] > numeros[j]) {
                    temp = numeros[i]
                    numeros[i] = numeros[j]
                    numeros[j] = temp
                }
            }
        }

        escreva("Seus números da sorte: ")
        para (i = 0; i < 6; i++) {
            escreva(numeros[i], " ")
        }
        escreva("\n\nBoa sorte! 🍀")
    }
}
```

📌 **Explicação:**

•Usa um vetor para armazenar os 6 números
•Laço `para` para gerar cada número
•Laço `faca-enquanto` para garantir números únicos
•Verifica se o número já existe antes de aceitar
•Ordena os números em ordem crescente

🎯 Exercício 3: Jogo dos Palpites

Enunciado: Elabore um jogo chamado "Jogo dos Palpites". Dois jogadores farão palpites de números entre 0 e 100. O jogador que chegar mais perto do valor aleatório ganha.

Solução:

```
programa {
    inclua biblioteca Util

    funcao inicio() {
        inteiro numero_secreto, palpite1, palpite2
        inteiro diferenca1, diferenca2

        escreva("=== JOGO DOS PALPITES ===\n")

        numero_secreto = Util.sorteia(0, 100)

        escreva("Jogador 1, digite seu palpite (0 a 100): ")
        leia(palpite1)
        escreva("Jogador 2, digite seu palpite (0 a 100): ")
        leia(palpite2)

        escreva("\n--- RESULTADO ---\n")
        escreva("Número secreto: ", numero_secreto, "\n")
        escreva("Palpite do Jogador 1: ", palpite1, "\n")
        escreva("Palpite do Jogador 2: ", palpite2, "\n\n")

        // Calcula as diferenças
        se (palpite1 > numero_secreto) {
            diferenca1 = palpite1 - numero_secreto
        } senao {
            diferenca1 = numero_secreto - palpite1
        }
```

```
        se (palpite2 > numero_secreto) {
            diferenca2 = palpite2 - numero_secreto
        } senao {
            diferenca2 = numero_secreto - palpite2
        }

        // Decide o vencedor
        se (diferenca1 < diferenca2) {
            escreva("🏆 JOGADOR 1 VENCEU! 🏆\n")
            escreva("Diferença: ", diferenca1)
        } senao se (diferenca2 < diferenca1) {
            escreva("🏆 JOGADOR 2 VENCEU! 🏆\n")
            escreva("Diferença: ", diferenca2)
        } senao {
            escreva("🤝 EMPATE! Ambos acertaram igualmente! 🤝")
        }
    }
}
```

📌 **Explicação:**

•Gera um número secreto aleatório
•Recebe os palpites dos dois jogadores
•Calcula a diferença absoluta de cada palpite
•Compara as diferenças e declara o vencedor

📊 Exercício 4: Cálculo IMC com Validação

Enunciado: Desenvolva um programa para calcular o IMC (Índice de Massa Corporal). Utilize um laço de repetição para validar se o usuário digitou valores de peso e altura maiores que 0.

Solução:

```
programa {
    funcao inicio() {
        real peso, altura, imc

        escreva("=== CÁLCULO DO IMC ===\n")
```

```
    // Validação do peso
    faca {
        escreva("Digite seu peso (kg): ")
        leia(peso)
        se (peso <= 0) {
            escreva("❌ Peso deve ser maior que 0!\n")
        }
    } enquanto (peso <= 0)

    // Validação da altura
    faca {
        escreva("Digite sua altura (m): ")
        leia(altura)
        se (altura <= 0) {
            escreva("❌ Altura deve ser maior que 0!\n")
        }
    } enquanto (altura <= 0)

    // Cálculo do IMC
    imc = peso / (altura * altura)

    escreva("\n=== RESULTADO ===\n")
    escreva("Seu IMC: ", imc, "\n")

    // Classificação
    se (imc >= 16 e imc <= 16.9) {
        escreva("Classificação: Muito abaixo do peso")
    } senao se (imc >= 17 e imc <= 18.4) {
        escreva("Classificação: Abaixo do peso")
    } senao se (imc >= 18.5 e imc <= 24.9) {
        escreva("Classificação: Peso normal 🎉")
    } senao se (imc >= 25 e imc <= 29.9) {
        escreva("Classificação: Acima do peso")
    } senao se (imc >= 30 e imc <= 34.9) {
        escreva("Classificação: Obesidade Grau I")
    } senao se (imc >= 35 e imc <= 40) {
        escreva("Classificação: Obesidade Grau II")
    } senao se (imc > 40) {
        escreva("Classificação: Obesidade Grau III")
    } senao {
        escreva("Classificação: Fora da tabela de referência")
    }
}
```

```
}
```

📌 **Explicação:**

•Usa `faca-enquanto` para validar peso > 0
•Usa `faca-enquanto` para validar altura > 0
•Calcula o IMC com a fórmula: peso / (altura × altura)
•Classifica o resultado com `se-senao se`

📐 Exercício 5: Tabuada Interativa

Enunciado: Utilizando o laço de repetição PARA, desenvolva um programa que faça uma tabuada. Leia um número de 1 a 10 e exiba a tabuada completa.

Solução:

portugol

```
programa {
    funcao inicio() {
        inteiro numero

        escreva("=== TABUADA INTERATIVA ===\n")

        // Validação do número
        faca {
            escreva("Digite um número de 1 a 10: ")
            leia(numero)
            se (numero < 1 ou numero > 10) {
                escreva("❌ Digite um valor entre 1 e 10!\n")
            }
        } enquanto (numero < 1 ou numero > 10)

        escreva("\n=== TABUADA DO ", numero, " ===\n")

        // Laço PARA para gerar a tabuada
        para (inteiro i = 1; i <= 10; i++) {
            escreva(numero, " x ", i, " = ", numero * i, "\n")
        }

        escreva("\nFim da tabuada!")
    }
```

}

📌 **Explicação:**

•Valida a entrada do usuário (1 a 10)
•Usa `para` com contador de 1 a 10
•Calcula e exibe cada linha da tabuada

---

4. QUADRO RESUMO

Conceito	Sintaxe	Uso Principal
Importar biblioteca	`inclua biblioteca Util`	Usar funções da biblioteca
Número aleatório	`Util.sorteia(min, max)`	Gerar valores aleatórios
Pausa	`Util.aguarde(ms)`	Aguardar um tempo
Tempo decorrido	`Util.tempo_decorrido()`	Medir tempo de execução
Laço ENQUANTO	`enquanto (condicao) { }`	Repetir enquanto condição for verdadeira
Laço FAÇA-ENQUANTO	`faca { } enquanto (condicao)`	Executar pelo menos uma vez
Laço PARA	`para (int i = 1; i <= n; i++) { }`	Repetir número definido de vezes

---

5. DICAS IMPORTANTES

1.**Sempre importe a biblioteca Util** antes de usar `Util.sorteia()`
2.Use **FAÇA-ENQUANTO** quando precisar executar o bloco pelo menos uma vez (validação de dados)
3.Use **ENQUANTO** quando a repetição pode não ocorrer (verificação prévia)
4.Use **PARA** quando souber exatamente quantas repetições serão necessárias
5.Para gerar números sem repetição, verifique se o número já foi gerado antes

---

📌 **Agora é sua vez!** Copie os códigos para o Portugol Webstudio , execute e experimente modificá-los para criar seus próprios programas!